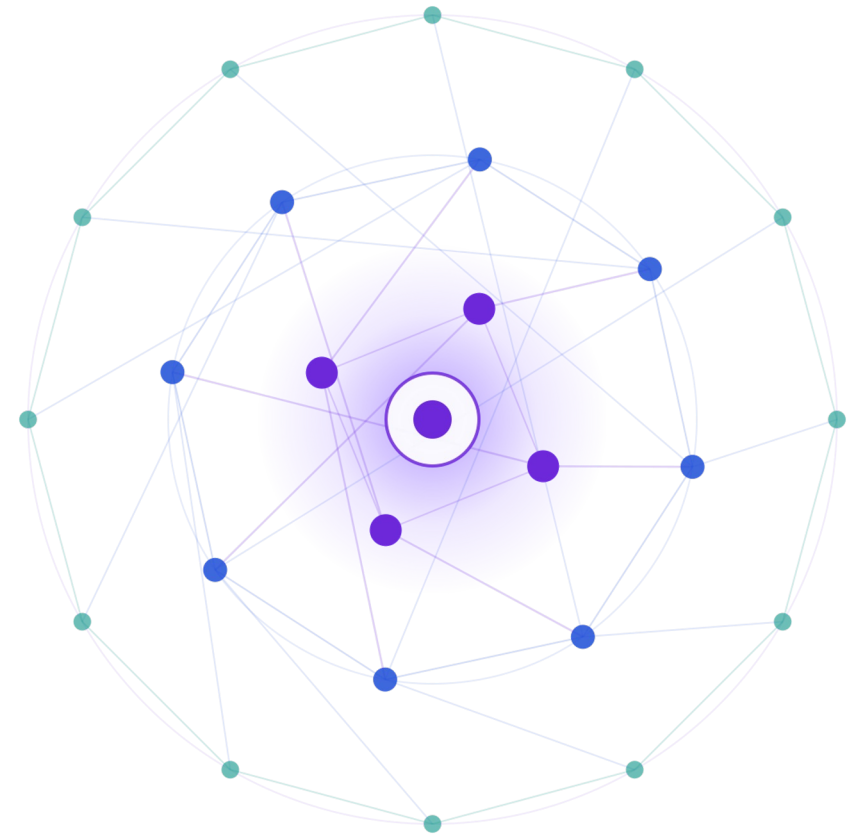


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID: **1 • Agentic Code Intelligence**
- Team Name: **floppydisk**
- College: **SRM Institute of Science and Technology**
- Members:
 - Akshith Rajesh** ar1870@srmist.edu.in
 - Sanjeev TS** st4469@srmist.edu.in
 - D Prethiv Sriman** dz1069@srmist.edu.in
 - Akhilan VTM** am7508@srmist.edu.in
- GitHub: **github.com/wtfPrethiv/seraph**



Seraph · query-adaptive, version-aware code retrieval

Theme: Agentic Code Intelligence

THE PROBLEM

Given a library of code and a question in plain English, rank the snippets by relevance, fast, on CPU, and across changing versions.

Thousands of snippets

Too many and too long to paste into any LLM context window.

Long, noisy questions

Problem statements mix a story, constraints and worked examples.

Code keeps changing

Every commit moves functions; agents still need old versions.

Our answer: Seraph, a retrieval engine that scores code several ways, fuses the scores, indexes every Git version and plugs into a coding agent.

Existing Solutions & Gaps

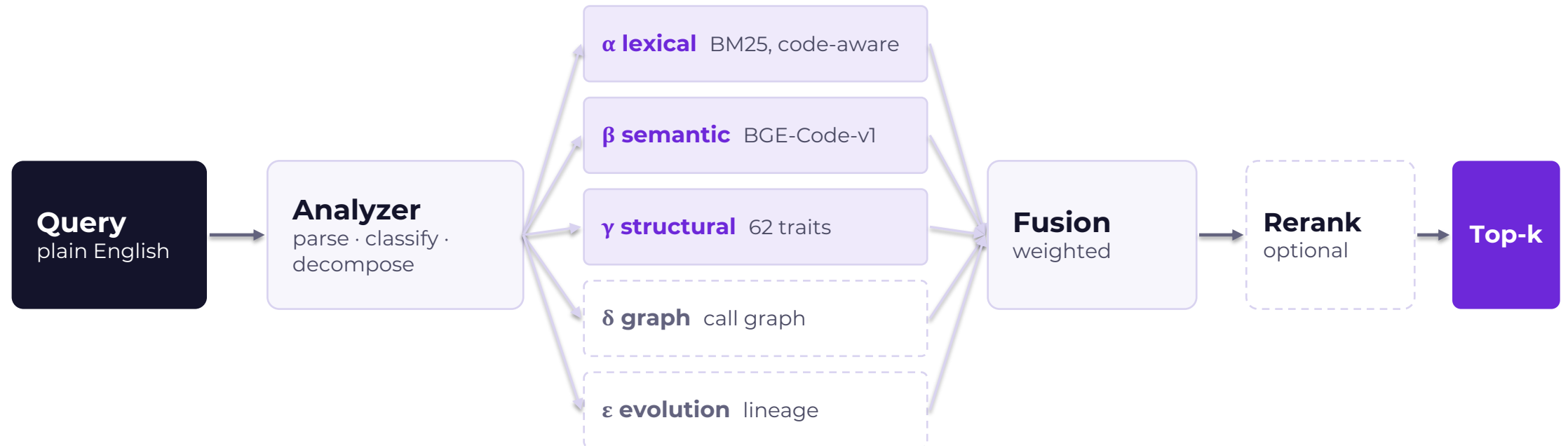
HOW AGENTS FIND CODE TODAY

Approach	Good at	Gap Seraph closes
grep / ripgrep loops in coding agents	Exact names and strings	Many tool calls when the name is unknown; no ranking
Embedding-only code indexes	Search by meaning	One version of the code; no history, no structure signal
Ask an LLM to rank snippets	Deep reading	Context window and latency limits on thousands of snippets
Generic text embedders	Easy to plug in	Weak on code: 0.16 to 0.51 nDCG@10 on stdin-style problems

Generic-embedder range from our own sweep (gte-modernbert, CodeSage, CodeRankEmbed, jina-code-v2) on the dev_stdin split.

Our Solution & Architecture

HOW SERAPH RANKS CODE



Every view scores every candidate; fusion mixes them with weights 0.16 / 0.64 / 0.2, tuned on train/dev only. Dashed views run on real repositories, where there is a call graph and a history.

Index: tree-sitter symbol chunks per Git commit in SQLite, a call, import and inheritance graph, and symbol lineage that follows functions through renames and moves.

Demo & Product Walkthrough

SERAPH INSIDE THE MORPHEUS CODING AGENT

1

Ask in plain English

"where are MCP server
processes started?"

Top hit
McpConnection.connect,
search 2 ms

2

Switch versions

`index a new commit`

741 of 749 chunks reused,
186 ms

3

Search all versions

`search_history`

identical code shown
once, tagged +N versions

4

Compare with grep

`/vs <question>`

Seraph's #1 is grep hit
#277 of 399

```
$ /vs where are mcp server processes started
◇ SERAPH search 2 ms    1 McpConnection.connect  src/tools/mcp.ts:317
◇ GREP 41 ms · 399 hits  1 README.md:49    2 README.md:51 ...
```

Demo video: drive.google.com/drive/folders/1TbShN44FSyemdtEXjca1lKWpaW50liuU

Tools and Tech Stack Used

EVERYTHING RUNS LOCALLY

Retrieval

Python 3.12 · uv · bm25s · sentence-transformers · PyTorch (CPU, CUDA, Apple MPS) · faiss

Models

BGE-Code-v1 (submitted) · Qwen3-Embedding · EmbeddingGemma · scikit-learn and LightGBM for the query models

Code parsing

tree-sitter grammars for Python, TypeScript, JavaScript and Go

Storage

SQLite per-commit index · content-hashed embedding cache

Evaluation

MTEB 2.x (AppsRetrieval, CoIR) · own metrics cross-checked to within 2e-6

Agent integration

MCP server (FastMCP) · Morpheus terminal agent (TypeScript, Ink) · Cloudflare Workers + OpenRouter for the capped demo model

Impact & Use Case

WHO IT HELPS

Coding agents

One search call returns ranked functions with paths and line ranges, instead of many grep-and-read rounds.

Bug hunting across releases

Compare two versions: changed symbols, dependencies and commits; follow a function through renames.

Large, fast-moving repos

Django: 2,899 files indexed in 31 s; the next release re-indexes in 1.3 s, reading only 38 changed files.

Runs on a laptop CPU

No API calls. CPU only: about 0.9 s per new question with BGE-Code-v1, 0.15 s with EmbeddingGemma-300M; precomputed vectors skip the corpus build.

Innovation, Results & Limitations

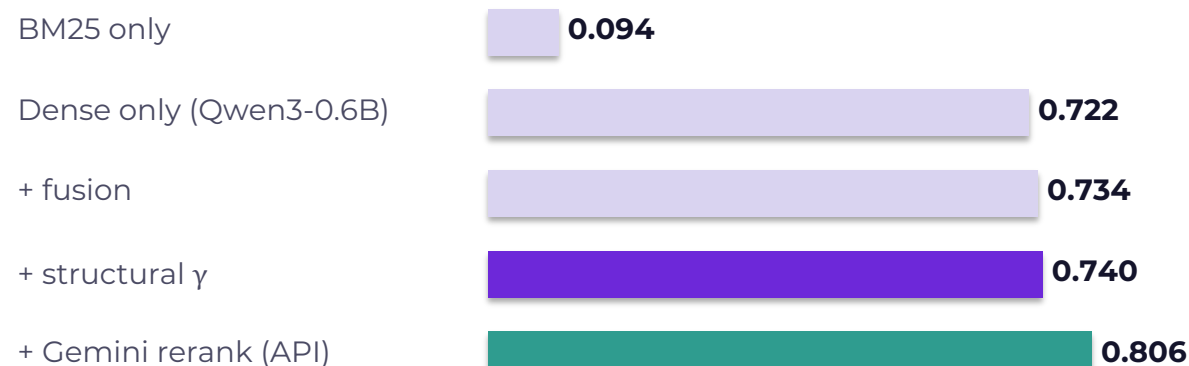
COIR APPSRETRIEVAL · TEST SPLIT · 3,765 QUERIES

0.9770

nDCG@10 · pipeline + BGE-Code-v1

MRR@10 0.970 · Recall@10 0.997

Ablation on dev_stdin, nDCG@10 (Qwen3 pipeline)



Innovation

Structural γ view predicts the traits of the answer (stdin, grids, BFS...) from the question

Versioned index with chunk reuse, rename-aware symbol lineage and a cross-file call graph

Same pipeline, swappable embedder: 0.749 with Qwen3, 0.977 with BGE-Code-v1

Limitations

BGE-Code-v1 likely saw overlapping public data; test matches its reported 98.08

With BGE the extra views add nothing on APPS (0.9795 alone); test split was run three times

Graph and lineage views are built but not benchmarked: APPS has no call graph or history

A Tough Query, Ranked

DEV_STDIN EXAMPLE · THE ANSWER SHARES NO KEY WORDS WITH THE QUESTION

Query (q1333)

Consider an infinite full binary tree. For a node labelled v , its left child is $2*v$ and its right child $2*v+1$; the root is 1.

For each query $i\ j$, print the length of the shortest path between node i and node j .

$1 \leq N \leq 10^5, \quad 1 \leq i, j \leq 10^9$

Keyword search (BM25)

```
1 # cook your dish here
2 # cook your dish here
3 #!/usr/bin/env python3
```

Wrong answers. The right solution is not in its top 100.

Seraph, rank 1: correct

```
t=eval(input())
for _ in range(t):
    i,j=map(int,input().split())
    bi=bin(i)[2:]
    bj=bin(j)[2:]
    ...
    print(len(bi)-k+len(bj)-k)
```

It never says tree or path: it compares binary labels.

What's Next

ROADMAP

01

Local reranker

Gemini listwise rerank gave +6.6 pts but costs 4.3 s per query; a local cross-encoder keeps it offline.

02

SeraphBench

A repo-level benchmark from real Git histories to measure the graph, evolution and dedup views.

03

Adaptive weights

A per-query oracle sits 4.4 to 5.7 pts above static weights; mixed query types should unlock it.

04

Agent benchmark

Measure how many tool calls and tokens Morpheus saves with Seraph compared with grep on real coding tasks.

Brownie Points (Differentiation)

WHAT GOES BEYOND THE SCREENING TASK

Version-aware by design

Search any version or all of them; functions are followed through renames and moves, and compare shows what changed between two refs.

Built into a real agent

Seven MCP tools (search, history, symbols, dependencies, compare) used by Morpheus, with ranked result cards.

Zero-setup demo

Morpheus runs on a capped cloud model when no key is set, and says so on screen.

Honest evaluation

Metrics cross-checked against MTEB to within 2e-6; train, dev and test kept apart; every run logged with its config hash.

Checklist - Updated on Public GitHub

SAMSUNG



Working prototype code: public GitHub repo github.com/wtfPrethiv/seraph

Y

README with reproducible setup instructions Results, setup, CLI, MCP and Morpheus sections

Y

Demo video, max 5 minutes drive.google.com/drive/folders/1TbShN44FSyemdtEXjca1lKWpaW50liuU

Y

Presentation file (PPT or PDF) This deck

Y

AppsRetrieval result JSON in a GitHub Release tag PRISM_GENAI_HACKATHON_Y2026

Y

AI usage disclosure docs/ in the repo

Y

SAMSUNG



Thank you

github.com/wtfPrethiv/seraph · github.com/projectakshith/morpheus

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India